

Project 03 — JavaScript & Interactivity

Breathing Life into the Browser

Summary: This document defines the subject matter for Project 03: Modern JavaScript syntax, variables, data types, functions, conditional control flow, DOM tree traversal and mutation, event-driven architecture, and browser Developer Tools debugging.

Version: 1.0

Contents

I	Instructions	2
II	AI & Peer-Learning Guidelines	3
III	Foreword	4
IV	Exercise 00: Variables & DevTools Console	5
V	Exercise 01: Functions & Conditional Branching	6
VI	Exercise 02: DOM Selection & Modification	7
VII	Exercise 03: Events & Event Listeners	8
VIII	Exercise 04: Interactive Mini-App	9
XIV	Submission and Peer-Evaluation	10

Chapter I: Instructions

- These exercises are carefully arranged in order of difficulty, from easiest to hardest. We will not consider a successfully completed harder exercise if an easier one is not fully functional.
- Ensure that you have the appropriate permissions and correct directory structure for all your files.
- You must follow the exact submission procedures and naming conventions for every exercise.
- Your exercises will be checked and graded by your fellow classmates during peer evaluations.
- Additionally, your exercises may be validated by automated evaluation scripts. The evaluation is rigorous and strict. To avoid unpleasant surprises, be as thorough as possible.
- You must not leave any extraneous files in your directories other than those explicitly specified in the subject.
- Have a question? Ask the peer on your right. If not, try the peer on your left.
- Your reference guides are official documentation (MDN, man, Git docs) and verified technical resources.
- Examine the examples and edge cases carefully. They contain critical details.

— *Did you check with your right-side neighbor?* —

Chapter II: AI Instructions

The Web Piscine is foundational. Your primary mission is to build solid problem-solving reflexes, genuine technical autonomy, and collaborative peer learning.

In the modern tech ecosystem, generative AI shortcuts are tempting. However, shortcuts prevent the formation of neural connections required for deep engineering intuition.

Key Rules on AI Usage:

- Apply your own reasoning to each exercise before consulting external AI tools.
- Never ask an AI to generate solutions directly for you. You must write every line of code yourself.
- During peer evaluation and exams, you will have to explain every architectural decision and command. If you cannot explain your code, you will receive zero points.
- Good practice: You are confused about a concept like Flexbox or Git staging. You discuss it with your peer for 10 minutes, sketch it on paper, and it clicks.
- Bad practice: Copying code without understanding. You fail the peer defense because you cannot answer conceptual questions.

— *Learning is not knowing the answer; it is developing the ability to find it.* —

Chapter III: Foreword

HTML gives structure. CSS gives presentation. JavaScript gives life.

Created in 1995, JavaScript is the programming language that powers dynamic client-side interactivity across every modern web browser. Through the Document Object Model (DOM) and event listeners, you will transform static pages into reactive software.

— The browser is your runtime; the console is your guide. —

Chapter IV — Exercise 00: Variables & Types

Directory: ex00/

Files to Submit: index.html, script.js

Authorized: let, const, console.log, typeof

First steps into programming logic.

- Create script.js and link it to index.html using defer.
- Declare a const string for your name, a let number for your age, and a let boolean for coding status.
- Log welcoming output, increment your age, and inspect data types using typeof.
- Open browser Developer Tools (F12) and inspect the Console output.

! The legacy keyword var is strictly forbidden.

— Press F12 and inspect your console. —

Chapter V — Exercise 01: Functions & Conditions

Directory: ex01/

Files to Submit: index.html, script.js

Authorized: function, if/else, ===, &&, ||

Decision-making algorithms.

- Write a function `calculateGrade(score)` returning letter grades (A, B, C, D, F) with input validation.
- Write a function `checkAccess(age, hasTicket)` returning a boolean based on multiple logical criteria.
- Test both functions in console with boundary cases (e.g. 90, 89, 0, 100, invalid inputs).

! Always use strict equality (`===`) instead of loose equality (`==`).

— A function must cleanly return values for all paths. —

Chapter VI — Exercise 02: The Live DOM Tree

Directory: ex02/

Files to Submit: index.html, script.js, style.css

Authorized: getElementById, querySelector, textContent, style, classList

Manipulating the page in real-time.

- Select HTML elements dynamically using `document.getElementById` and `document.querySelector`.
- Change text content using `.textContent`.
- Alter styling properties and toggle CSS classes (`.active`) using `.classList.add()`.

! Avoid dangerous `innerHTML` for plain text insertions.

— The DOM is a live tree of objects in browser memory. —

Chapter VII — Exercise 03: Event-Driven Web

Directory: ex03/

Files to Submit: index.html, script.js, style.css

Authorized: addEventListener, click, state variables

Responding to user actions.

- Build an interactive counter widget with Increment, Decrement, and Reset buttons.
- Bind click events using addEventListener without inline HTML handlers.
- Mutate state (let count = 0), update DOM text, and dynamically alter text colors (green for positive, red for negative).

! Inline onclick="..." attributes in HTML are strictly prohibited.

— Pass function references to addEventListener, not function calls. —

Chapter VIII — Exercise 04: Interactive App

Directory: ex04/

Files to Submit: index.html, script.js, style.css

Authorized: createElement, appendChild, remove, keydown

Synthesizing logic, events, and DOM.

- Build a Dynamic Task / Note Manager application.
- Allow users to add tasks via button click or Enter key press.
- Create dynamic li elements with document.createElement, append delete buttons, and allow marking tasks as completed.
- Keep a total task counter in sync with all additions and deletions.

— *Inspect network and console tabs to ensure error-free execution.* —

Chapter XIV: Submission & Evaluation

- Submit your assignment to your Git repository as instructed. Only the work pushed to your repository branch will be evaluated during defense.
- Double-check your directory structure, file permissions, and exact file names before final submission.

! You must submit only the files explicitly required by the project instructions.

- Prepare to demonstrate your code live, explain your logic line-by-line, and answer questions about core concepts.

— RTFM and verify your work before peer defense! —